

Automated Circuit Discovery for NIDS

January 24, 2026

Beyond the Black Box: ACDC to Reveal the Causality of Network Intrusions

Presented by: **Anbuselvi S**

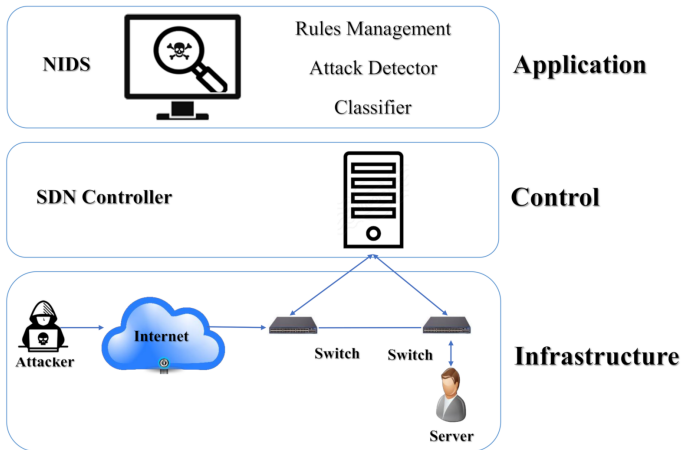
Presentation Outline

- 1 Motivation & Background
- 2 Technical Framework
- 3 Research Applications
- 4 SOC-Oriented Integration
- 5 Conclusion

Motivation: The Interpretability Gap in Network Security

- Modern network security systems increasingly rely on **deep learning** models for automated threat detection.
- **The Black-Box Problem:** These models generate alerts, but their **internal decision logic** is opaque to human analysts.
- **Operational Impact:** Security teams face **high false-positive rates**.
- **Key Challenge:** Understanding *why* a model raises an alert, not just *that* it raises one.
- Motivation: **causal interpretability** > correlational explanations. [1, 4]

Network Intrusion Detection System (NIDS)



High-level architecture of a Network Intrusion Detection System. [2]

Interpretability Methods in NIDS: A Comparison

Aspect	Feature Attribution (SHAP / LIME)	Rule-Based XAI	ACDC (Proposed)
Explanation Level	Input features	Human-defined rules	Internal computational components
Causality	Correlational	Assumed by rules	Causal (via activation patching)
Model Internals	Ignored	Not applicable	Explicitly analyzed
Attack-Specific Logic	Weak	Manual	Automatically extracted
SOC Usefulness	Limited	High but rigid	High and operationally actionable
Forensic Value	Low	Moderate	High (circuit-level audit)

Applying ACDC to NIDS: Core Idea

- In NIDS, the target behavior is an **attack detection decision** (e.g., SYN flood, scan).
- ACDC treats the trained NIDS as a **signal-processing system** mapping traffic statistics to decisions.
- The goal is to identify the **minimal internal computation**.
- The extracted circuit reveals the **features and interactions** that causally drive detection. [2, 3]

High-Level System Architecture

- **Data Plane:** Real-time packet ingestion and feature engineering.
- **Inference Plane:** Modular neural Network Intrusion Detection System.
- **Diagnostic Plane:** On-demand ACDC engine for causal analysis.
- Planes are **decoupled** to preserve throughput and reliability.

Interpretability is treated as a diagnostic service, not an inline requirement.

[1]

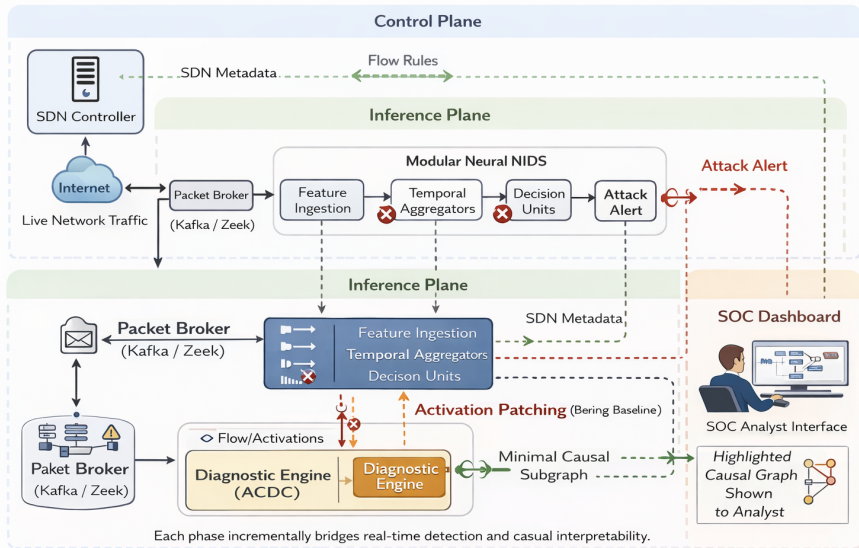
Data Plane: Ingestion and Feature Engineering

- **Packet Broker:** Zeek / Suricata tap live network traffic.
- **Stream Processor:** Kafka or Spark Streaming aggregates packets into flows.
- **Computed Features:**
 - Packet rate
 - Protocol flags
 - Byte and payload statistics
- **State Store:** Redis or Cassandra maintains temporal context (e.g., SYN count over last 10 seconds).

- Model implemented as a **modular computational graph**.
- Each logical feature group corresponds to a **distinct sub-network**.
- Enables node-level intervention required by ACDC.
- **Shadow Mode:**
 - Model runs continuously.
 - High-risk alerts trigger logging of internal activations.

- Activated **only for high-confidence alerts**.
- Receives:
 - Flow identifier
 - Full internal activation trace
- **Activation Patching Controller:**
 - Injects baseline (benign) activations.
 - Performs counterfactual tests.
- **Circuit Extractor:**
 - Prunes non-causal edges.
 - Outputs minimal causal subgraph.

System Architecture: ACDC-Enabled NIDS



Automated Circuit Discovery Loop

- ➊ Input: Malicious flow x_m , benign flow x_b .
- ➋ Forward pass: record all node activations.
- ➌ Patch each edge with benign activations.
- ➍ Measure output divergence (KL divergence).
- ➎ If divergence $< \epsilon$, mark edge as non-causal.
- ➏ Recursively prune graph.

Result: a minimal circuit sufficient to explain the alert.

Step A: Task and Dataset Setup

To find a circuit, we must define the behavior:

Task Definition

Detecting SYN Flood attacks within the UNSW-NB15 dataset.

- **Behavior-Preserving Dataset:** Malicious traffic samples
- **Behavior-Breaking Dataset:** Normal baseline traffic

Behavior is quantified using the logit difference between Attack and Normal classes.

Datasets for Attack-Specific Circuit Discovery

(a) Basic statistics of datasets

Dataset	Labels	Features	Samples
CICIDS-2017	7	78	2.78M
RoEduNet-SIMARGL2021	3	29	31.43M
NSL-KDD	5	41	148K

(b) Distribution of samples across attack types

Dataset	Normal	DoS	PortScan	Brute	Web	Bot	Probe	U2R
CICIDS-2017	84.4%	9.1%	5.7%	0.5%	0.16%	0.07%	—	—
RoEduNet-SIMARGL2021	62.2%	24.5%	13.3%	—	—	—	—	—
NSL-KDD	48.0%	35.0%	—	—	—	—	10.1%	0.47%

These datasets enable attack-specific behavior isolation and evaluation of circuit faithfulness under realistic class imbalance.

Step B: Pruning Criterion (τ)

ACDC removes an internal connection ($w \rightarrow v$) if it does not causally influence the attack detection outcome.

$$D_{KL}(G \parallel H_{\text{new}}) - D_{KL}(G \parallel H) < \tau$$

- G : Full NIDS model predicting attack vs. normal traffic.
- H : Current candidate circuit under evaluation.
- D_{KL} : Change in output probability distribution.

KL divergence ensures the circuit preserves the **decision logic**, not merely classification accuracy.

Step C: Activation Patching

- Internal activations from **corrupted (benign) traffic** are injected into the forward pass of **malicious traffic**.
- If the attack prediction remains unchanged, the patched connection is **not causally required** for detection.
- This allows ACDC to distinguish **true attack logic** from incidental correlations.
- **Example:** replacing SYN packet-rate activations with statistics from normal background traffic.

Research: Generalization vs. Spurious Correlation

- Does the trained NIDS learn **attack-relevant traffic patterns** (e.g., TCP flags, packet-rate bursts, temporal structure)?
- Or does it rely on **incidental identifiers** (e.g., source IPs, ports, or dataset-specific artifacts)?
- ACDC reveals whether the extracted circuit encodes **robust attack semantics** or shortcut-based correlations. [1]

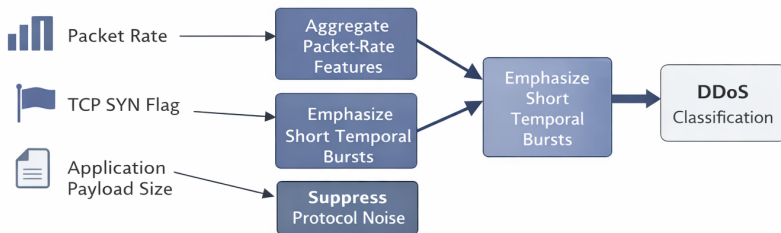
Example: Circuit Discovery for DDoS Detection

- Consider a trained NIDS tasked with detecting **volumetric DDoS attacks**.
- ACDC isolates a **minimal detection circuit** that:
 - aggregates packet-rate signals across short time windows,
 - emphasizes sudden temporal traffic bursts,
 - suppresses protocol-specific and benign background noise.
- This circuit constitutes the **causal computation** sufficient for triggering a DDoS alert.

ACDC-Based Circuit Extraction for DDoS Attacks

Example: Circuit Discovery for DDoS Detection

ACDC isolates a DDoS-specific circuit within the trained NIDS



Logic Redundancy Discovery: The OR-Gate Problem

- **Redundancy:** A NIDS may learn multiple internal pathways that independently detect the same attack.
- **Pruning Risk:** If removing one pathway does not affect performance, ACDC may prune both during causal testing.
- **Interpretation:** This reflects redundancy in detection logic—not model failure.
- **Research Challenge:** Handling redundant causal pathways remains an open problem in circuit discovery.

Scalability via an Asynchronous Forensic Pipeline

- **Decoupled Design:** ACDC functions as a diagnostic service, separate from inline detection.
- **Parallel Execution:** Real-time inference and causal circuit discovery run in independent planes.
- **No Impact on Inline Detection:** Activation patching is isolated, preserving throughput and latency.
- **Efficient Discovery:** Attribution patching reduces computational cost.
- **Flow:** *Detect* → *Trigger* → *Explain*.

Traditional Alert: *Anomaly detected*

ACDC-Enhanced Alert

Logic: Observed high SYN count combined with low data payload patterns.

Causal confidence: 99.2%.

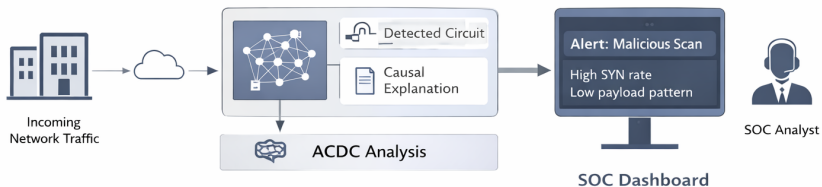
Circuits are summarized, not shown raw, to analysts.

- Visual graph with **causal path highlighted**.
- Node-level attribution replaces feature-only explanations.
- Automatic rule translation:

"This alert was triggered by a sustained spike in byte counts while TCP flags remained in a SYN-SENT state."

SOC Integration: System Design

SOC-oriented Explainable Alerts



Technical Implementation Roadmap

Phase	Task	Tools / Technologies
Phase 1	Model Instrumentation	Hooking internal layers to capture node activations using PyTorch / TensorFlow forward hooks
Phase 2	Circuit Discovery Loop	Implementing the τ -based pruning criterion using KL divergence to remove non-causal edges
Phase 3	Streaming Integration	Connecting the ACDC engine to live traffic pipelines via Kafka, Zeek, or Suricata for asynchronous forensics
Phase 4	SOC API Layer	Building REST APIs that expose causal circuits to the SOC dashboard as structured, auditable graphs

Each phase incrementally bridges real-time detection and causal interpretability.

Circuit Level Debugging for NIDS Models

- **The Bottleneck:** Traditional NIDS are "Black Boxes"—fixing one error often requires retraining the entire model.
- **ACDC-Enabled Debugging:**
 - **Pinpoint:** Identify the specific sub-circuit responsible for a false alert.
 - **Causal Validation:** Confirm the error using activation patching with "clean" baseline traffic.
 - **Targeted Update:** Refine only the implicated nodes, leaving the rest of the model's logic intact.
- **Operational Value:**
 - Faster time-to-patch for SOC teams.
 - Eliminates the risk of "catastrophic forgetting" during global retraining.

Evaluation: Quantifying Model Trust

- **How do we know the refined circuit is reliable?**
- **Faithfulness (D_{KL}):** Does the simplified circuit still "think" like the full model?
- **Robustness (Accuracy):** Does detection remain high after we prune the "noise"?
- **Interpretability (Sparsity):** Is the circuit small enough for a human to audit in under 60 seconds? [2]

- **Recovery Quality:** Proof that the circuit captures the *actual* attack signatures, not just random noise.
- **Corrupted Traffic Stress Test:**
 - Does the circuit break if we corrupt a malicious feature?
 - If yes, the circuit has successfully identified a **causal feature**.
- **Q1/Q2 Logical Integrity:**
 - **Q1 (Sufficiency):** Is the circuit enough to detect the attack?
 - **Q2 (Minimality):** Does removing any more edges break the detection?

Building Operational Trust

- **Auditability:** ACDC provides a causal explanation for each intrusion detection decision.
- **Accountability:** Enables post-incident review and justification in regulated and security-critical environments.
- **Inspectability:** Improves analyst confidence by making AI-driven decisions transparent rather than opaque.

- Framed network intrusion detection as a **mechanistic interpretability problem**, rather than a black-box prediction task.
- Applied Automatic Circuit Discovery (ACDC) to extract **causal, attack-specific computational circuits** from trained NIDS models.
- Demonstrated that circuit-level explanations enable **auditable and trust-oriented decision support** for SOC operations.

Future Directions: The Roadmap to Trusted NIDS

- **Vision:** Moving from black-box detection to an auditable, interactive security ecosystem.
- **Real-Time Scalability**
Approximate ACDC and attribution patching for high-throughput, active network environments.
- **Advanced Threat Discovery**
Zero-day behavioral analysis and resolving "OR-Gate" redundancy in overlapping attack logic.
- **The Human-in-the-Loop SOC**
Automated circuit-to-rule translation and interactive tools for AI-driven decision refinement.

- [1] Karim Ahmed, M. U. Akram, and Ayesha Javed. “Explainable Intrusion Detection Systems: A Survey”. In: *Journal of Network and Computer Applications* 191 (2021), p. 103163.
- [2] Arthur Conmy et al. “Towards Automated Circuit Discovery for Mechanistic Interpretability”. In: *arXiv preprint arXiv:2304.14997* (2023).
- [3] Clare Lyle, Johannes Kleiner, and Jacob Steinhardt. “Efficient Automated Circuit Discovery in Transformers Using Contextual Decomposition”. In: *arXiv preprint arXiv:2306.05427* (2023).
- [4] Yuwei Xin et al. “XAI-IDS: Towards Proposing an Explainable Artificial Intelligence Model for Intrusion Detection Systems”. In: *IEEE Access* 10 (2022), pp. 12345–12360.

Thank You!